

# Technická dokumentace: Analýza a složitost algoritmu pro hledání prvočíselných dvojčat

Patrik Mlčák

June 3, 2026

## 1 Analýza problému, datové typy a popis funkcí

---

### 1.1 Analýza problému

Úloha vyžaduje nalezení  $m$  prvních prvočíselných dvojčat  $(p, p + 2)$  takových, že  $p > n$ . Problém se dělí na tři logické části:

- Matematická část:** Nalezení a ověření prvočísel pomocí efektivního algoritmu (trial division do odmocniny).
- Ukládání dat:** Uložení menších čísel z nalezených dvojčat do dynamické datové struktury typu FIFO (*First-In, First-Out*) spojového seznamu.
- I/O operace:** Bezpečné načtení parametrů  $(m, n)$  a uložení struktury do textového souboru s následným uvolněním alokované paměti.

### 1.2 Návrh datových typů

- `long long`: Použit pro reprezentaci testovaných čísel  $(p, n)$ . Zabraňuje přetečení pro hodnoty  $n > 2^{31} - 1$ .
- `struct Node`: Uzel jednosměrně spojového seznamu. Obsahuje datovou složku (`long long value`) pro uložení hodnoty  $p$  a ukazatel (`struct Node *next`) na další prvek v paměti.
- `struct Queue` (ADT): Struktura reprezentující FIFO frontu. Zapouzdřuje ukazatel na začátek (`head` – pro odebírání), konec (`tail` – pro vkládání) a velikost fronty (`int size`).

### 1.3 Slovní popis funkcí (metod)

Metody ADT Fronty (`queue.c` / `queue.h`)

- `queue_init`: Inicializuje prázdnou frontu. Nastaví ukazatele `head` i `tail` na `NULL` a velikost na 0.
- `queue_enqueue`: Dynamicky alokuje (`malloc`) nový uzel (`Node`). Vloží do něj hodnotu a připojí jej na konec fronty (za aktuální `tail`), čímž dodržuje princip FIFO.
- `queue_dequeue`: Odebere uzel ze začátku fronty (z `head`). Uloží jeho hodnotu přes ukazatel, přesune `head` na další prvek, uvolní paměť uzlu (`free`) a vrátí úspěch.
- `queue_is_empty`: Konstantní operace vracející nenulovou hodnotu, pokud je fronta prázdná.
- `queue_traverse`: Nedestruktivní průchod frontou od `head` po `tail`. Na každý prvek aplikuje uživatelsky definovanou funkci předanou přes ukazatel (`callback`).
- `queue_free`: Bezpečnostní metoda zabraňující *memory leaks*. Cyklicky volá `queue_dequeue`, dokud není fronta zcela prázdná.

### Matematické funkce a hlavní logika (`main.c`)

- `is_prime`: Ověřuje prvočíselnost. Optimalizováno vynecháním sudých čísel a testováním dělitelnosti pouze do odmocniny ( $i \cdot i \leq n$ ).
- `find_twin_primes`: Generuje lichá čísla od  $n+1$  a pomocí `is_prime` hledá dvojčata. První číslo z úspěšného páru vkládá do ADT fronty.
- `read_input`: Validuje a parsuje argumenty příkazového řádku (`argc`, `argv`) a ošetřuje chybové vstupy.
- `write_output`: Otevře textový soubor a cyklickým odebíráním prvků z fronty (`dequeue`) do něj sekvenčně zapisuje výsledky.
- `print_to_console`: *Callback* funkce předávaná do `queue_traverse` sloužící k výpisu do konzole.

## 2 Složitost navrženého algoritmu a jeho omezení

---

### 2.1 Časová složitost

- **Operace nad ADT (Queue):** Zařazení (`enqueue`) a vyřazení (`dequeue`) jsou implementovány s asymptotickou složitostí  $\mathcal{O}(1)$ , jelikož udržujeme přímé ukazatele na `head` i `tail`. Průchod frontou (`traverse`) a zápis do souboru pracují v čase  $\mathcal{O}(m)$ .
- **Test prvočíselnosti (`is_prime`):** Algoritmus (*trial division*) testuje dělitele pouze do odmocniny z čísla  $N$ , časová složitost je tedy  $\mathcal{O}(\sqrt{N})$ .
- **Celková časová složitost hledání:** Závisí na hustotě výskytu prvočíselných dvojčat (což je v matematice neuzavřený problém). Pokud označíme  $k$  jako počet lichých čísel, která musíme otestovat, než najdeme  $m$  dvojčat, a  $P_{\max}$  jako největší testované číslo, je celková časová složitost  $\mathcal{O}(k \cdot \sqrt{P_{\max}})$ .

### 2.2 Prostorová (paměťová) složitost

- Prostorová složitost je  $\mathcal{O}(m)$ .
- Do spojového seznamu ukládáme přesně  $m$  uzlů (řešení). Paměť roste lineárně s počtem požadovaných dvojčat. Každý uzel zabírá v paměti konstantní velikost (8 bytů pro `long long` + 8 bytů pro ukazatel na 64-bit architektuře, tj. 16 bytů na uzel). Není alokována žádná zbytečná paměť pro mezivýpočty.

### 2.3 Omezení algoritmu (Limitations)

- **Omezení datových typů (Overflow):** Použitý typ `long long` omezuje maximální testované číslo na cca  $9.22 \times 10^{18}$ . V extrémním případě, kdy by se  $n$  blížilo tomuto maximu, by výpočet  $i \cdot i$  v cyklu `is_prime()` mohl přetéct přes maximální kapacitu typu `long long`, což by vedlo k nedefinovanému chování (*Undefined Behavior*).
- **Výpočetní (výkonnostní) omezení:** Metoda "*trial division*" je dostatečná pro běžná čísla (do řádu  $10^{12}$ ), ale pro kryptograficky velká  $n$  by byla extrémně pomalá. Pro reálné využití u gigantických čísel by bylo nutné algoritmus nahradit pravděpodobnostním testováním (např. Miller-Rabinův test).
- **Paměťové omezení haldy (Heap limit):** Vzhledem k dynamické alokaci (`malloc`) je hodnota  $m$  omezena pouze volnou operační pamětí počítače (RAM). Při extrémně vysokém  $m$  (v rádech desítek miliard) by došlo k vyčerpání paměti a `malloc` by vrátil `NULL` (což program našťestí bezpečně ošetřuje a nezhroutí se).